

MDP-Guided Reinforcement Learning for Cost-Aware LLM-Based Test Generation

Phung Trung Nguyen Nguyen¹, Hieu Le Minh¹, Thuan Mai Duc¹, Ngoc Khanh Tran Nguyen², and Xuan-Bach Le¹*

¹ Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam

{nguyen.nguyen938,hieu.lehieulhks}@hcmut.edu.vn

{thuan.maiduc2411,lexuanbach}@hcmut.edu.vn

² University of Science, VNU-HCM, Vietnam

22120378@student.hcmus.edu.vn

Abstract. LLM-based test generation is often framed as prompt design, but its operational core is sequential budget allocation: after each execution, a system must decide whether boundary generation, fuzzing, LLM synthesis, oracle refinement, repair-context retesting, or stopping is worth the next unit of budget. We present TestGenRL, an MDP-guided framework for cost-aware LLM test generation: it records evidence, masks infeasible or unaffordable actions, rewards failure discovery, coverage, uncertainty reduction, and repair-relevant evidence against cost, and trains an action-masked Double-DQN on the logged transitions. Because it is trained offline, a vanilla learner over-estimates actions under-represented in the buffer; we train it conservatively, which in our held-out evaluation removes this over-exploration while preserving regime-dependent adaptation. On a 74-subject Python benchmark (Gemini 2.5 Flash Lite caches) with a held-out test split, the value of planning depends on the workload regime. On the full-pilot split, Greedy, a contextual bandit (Bandit), and the conservative Double-DQN all recover the cheap boundary strategy, reaching 18.2% failure-found rate and 97.3% coverage. When duplicate penalties and action-family saturation make boundary tests less useful, the adaptive planners avoid Greedy’s over-commitment: Bandit and the Double-DQN reach 66.7% failure-found rate on stress subjects and 58.3%/55.6% on action-balanced subjects, versus Greedy’s 25.0% and 16.7%, by reallocating toward fuzzing. We deliver a concrete planning interface, and a calibration lesson, for when evidence changes which action is feasible, affordable, and useful.

Keywords: LLM test generation · Reinforcement learning · Markov decision processes · Budgeted testing · Fuzzing · Test repair

1 Introduction

Generated tests are useful only when they provide behavioral evidence a developer can trust. Modern LLM-based systems synthesize unit tests and assertions,

* Corresponding author.

fuzz inputs, exploit program context, and feed repair loops [19,3,4,2,26,7]. Once execution begins, however, the central question is no longer which prompt to issue, but what to execute *next*, given the failures already seen, the redundant or flaky tests, the obligations still uncovered, and the budget still available.

We treat LLM test generation as resource allocation rather than prompt engineering. The decision unit is not a token or a single test, but a generation action with prerequisites, costs, and uncertain downstream value: boundary tests are cheap and decisive early, fuzzing helps once simple cases saturate, oracle refinement matters only after assertions become ambiguous, and repair-context retesting is meaningless until a repair exists. The same intuition drives adaptive fuzzing, where seed scheduling and energy assignment are feedback-driven allocation problems [23,14,22]; TestGenRL brings that view to heterogeneous LLM-assisted testing. We therefore look beyond final failure-found rate: two policies may expose the same failures while spending very different LLM budget, exercising different obligations, or relying on different fuzzing. Such differences decide whether a suite is worth using, so we also report the action trace.

TestGenRL models test generation as a finite-horizon, budget-constrained Markov decision process. The state summarizes coverage, uncovered obligations, failures, counterexamples, repair events, oracle uncertainty, action history, and remaining budget; the eight top-level actions span boundary-test generation, property-based fuzzing, LLM instruction and assertion synthesis, oracle refinement, counterexample expansion, repair-context retesting, and stopping (Section 3). An action mask $m_t \in \{0, 1\}^{|A|}$ removes steps whose prerequisites are absent or whose cost cannot be paid. We train an action-masked Double-DQN [16,8] over this format. Trained offline from logged baseline trajectories, a vanilla learner over-estimates under-represented actions [5,11]; we control this with a conservative objective that recovers the cheap optimum where it suffices and keeps adaptation where it pays.

We evaluate TestGenRL on a 74-subject Python benchmark (HumanEval-style, MBPP-style, curated real-bug, stress, and action-balanced tasks) with Gemini 2.5 Flash Lite caches, training on a training split and reporting held-out results. On the full-pilot test split, Greedy, a contextual bandit (Bandit), and the conservative Double-DQN all reach 18.2% failure-found rate and 97.3% coverage by favoring boundary tests. On stress subjects, Bandit and the Double-DQN reach 66.7% versus Greedy’s 25.0%, and on action-balanced subjects 58.3% and 55.6% versus 16.7%, by reallocating toward fuzzing. Planning matters, in short, when the next useful action depends on what earlier actions revealed.

Contributions.

- We formulate LLM-based test generation as a budget-constrained MDP with masks for operational prerequisites and resource limits.
- We train a masked Double-DQN over a transition format for heterogeneous generation, oracle, fuzzing, repair, and stopping actions, and identify that offline training from logged baselines over-estimates under-covered actions; a conservative value objective [11] removes this over-exploration while preserving regime-dependent adaptation.

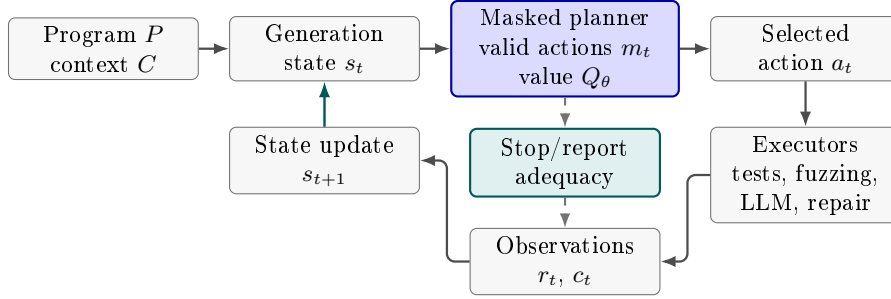


Fig. 1. MDP-guided reinforcement-learning loop for cost-aware test generation.

- We build a replayable Python prototype and show, on a held-out evaluation, when adaptive planning improves over myopic Greedy, and when cheap boundary tests already explain the outcome.

Section 2 describes the framework loop, Section 3 the budgeted MDP, Section 4 the protocol, Section 5 the evidence, and Sections 6–7 the implications and related work; a companion appendix gives expanded details.

2 Framework Overview

For a program P and context C , TestGenRL maintains a running record of what has already been tried and what evidence those attempts produced. At each step it masks invalid actions, selects a valid action, executes it, and records the new observations. The final product is not only a test suite, but an inspectable account of which obligations were exercised, which failures were exposed, which assertions remain uncertain, and which regions still look risky. Figure 1 summarizes this loop, in which the trained value function Q_θ scores the valid actions.

TestGenRL treats LLM test generation, assertion synthesis, fuzzing, counterexample expansion, and repair-context retesting as selectable executors rather than monolithic strategies. The design borrows from adaptive fuzzing [23,14,6], where feedback decides how much energy a seed or mutation receives; here it decides which testing mechanism receives the next unit of budget.

The loop stays operational because feasibility and cost are explicit. Actions carry prerequisites: counterexample expansion requires a failure input, repair-context retesting requires a repair event, and LLM actions require token budget. Masks enforce these constraints before scoring. Costs are also heterogeneous: boundary tests and fuzzing mostly consume execution budget, while LLM steps consume tokens and may add oracle uncertainty. Replay separates cache construction from policy evaluation, so planners face the same logged provider behavior and deterministic executor seeds.

3 Formalism

TestGenRL spends each additional unit of budget where it is expected to improve the validation record, and the formal model makes three commitments explicit. State must retain the facts that change the value of the next action (uncovered obligations, failures, duplicate tests, oracle uncertainty, remaining budget). Masks must enforce semantic prerequisites before learning, since counterexample expansion without a counterexample is invalid rather than merely low reward. Reward must connect outcome quality with cost, so a policy can prefer a cheaper action that produces equivalent evidence. The planner then observes state, removes infeasible actions, executes the selected action, and records evidence, reward, and cost.

3.1 Budget-Constrained MDP

For each program instance, TestGenRL defines a finite-horizon MDP whose transitions are revealed by execution rather than known in advance. The notation is standard, but the interpretation is operational: a state is a compact summary of the current test-generation log, an action is a concrete generator invocation, and the budget is a hard resource constraint rather than a soft preference.

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, R, \gamma, H, B).$$

Here, $\mathcal{S}$ contains evidence states, $\mathcal{A}$ contains generation actions, $\mathcal{P}(s_{t+1} | s_t, a_t)$ is unknown before execution, R scores evidence changes, γ discounts delayed outcomes, H is the horizon, and B is a normalized scalar budget. We maintain separate caps for LLM tokens, runtime, solver time, and fuzzing iterations. The objective uses $c_t = \text{tokens}/1000 + \text{runtime}/60 + \text{fuzz}/100 + \text{solver}/60$, and masks enforce each cap.

s_t encodes the generation log: obligation coverage, generated tests, failures, invalid/flaky/duplicate tests, counterexamples, oracle uncertainty, repair context, recent actions, costs, remaining budget, horizon, and LLM-output quality.

The action space has eight top-level actions ($|\mathcal{A}| = 8$): boundary-test generation, property-based fuzzing, LLM instruction synthesis, LLM assertion synthesis, oracle refinement, counterexample expansion, repair-context retesting, and stopping. These actions are intentionally coarse: executor parameters such as fuzzing duration and prompt size remain fixed, while the planner learns which action family to select. An action mask $m_t = m(s_t, b_t)$ disables impossible or unaffordable actions. Counterexample expansion requires a failure input, repair-aware retesting requires a repair event, oracle refinement requires uncertain oracle evidence, and LLM actions require remaining token budget. Stop is always valid. The mask prevents the value function from treating prerequisite violations as ordinary low-reward choices.

The action set covers deterministic evidence acquisition, LLM-assisted test and oracle synthesis, and later-stage exploitation of failures, uncertain assertions, or repair context. Its value is sequential: the right action can change once one of these facts appears.

3.2 Reward and Solver

The reward is shaped around evidence rather than raw test count. A useful action may expose a new failure, exercise a new obligation, reduce oracle uncertainty, or make repair overfitting visible. An unhelpful action spends budget on duplicates, flaky tests, or invalid outputs. The scalar reward below makes these tradeoffs comparable across heterogeneous executors:

$$r_t = \alpha M_t + \beta C_t + w_U U_t + \delta O_t + \eta Z_t - \lambda c_t - \rho D_t,$$

where M_t is newly discovered failures, C_t is new coverage (Section 4), U_t is oracle-uncertainty reduction, O_t is repair-overfitting evidence, Z_t is residual-risk reduction, and D_t counts invalid, flaky, or duplicate tests. The reported runs fix

$$\alpha = 10, \quad \beta = 1, \quad w_U = 1, \quad \delta = 10, \quad \eta = 1, \quad \lambda = 1, \quad \rho = 0.5, \quad \gamma = 0.95.$$

The evaluation section gives the remaining budget, cache, seed, and replay-training settings. The policy objective is

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right] \quad \text{s.t.} \quad \sum_{t=0}^{H-1} c_t \leq B.$$

Top-level actions are discrete, so the framework is compatible with masked Double DQN. The mask appears again in the target because learning should compare only actions that will be valid at the next evidence state. For transition $(s_t, a_t, r_t, s_{t+1}, m_t, m_{t+1}, c_t)$, the next-state mask defines

$$y_t = r_t + \gamma Q_{\bar{\theta}} \left(s_{t+1}, \arg \max_{a' : m_{t+1}(a')=1} Q_{\theta}(s_{t+1}, a') \right),$$

and the training loss over the logged buffer $\mathcal{D}$ is

$$L(\theta) = \mathbb{E}_{\mathcal{D}} \left[\ell_1^{\text{sm}}(Q_{\theta}(s_t, a_t) - y_t) \right] + \alpha_{\text{CQL}} \mathbb{E}_{\mathcal{D}} \left[\log \sum_{a \in \mathcal{A}} e^{Q_{\theta}(s_t, a)} - Q_{\theta}(s_t, a_t) \right],$$

where ℓ_1^{sm} is the smooth L_1 (Huber) loss and $\alpha_{\text{CQL}} = 0.5$ weights the CQL($\mathcal{H}$) regularizer [11], whose log-sum-exp spans all eight actions, masked ones included. Our Python prototype trains a masked Double-DQN on the logged transitions. The state-conditioned value function Q_{θ} is a small multilayer perceptron ($24 \rightarrow 64 \rightarrow 64 \rightarrow 8$) with ReLU activations for hidden layers and a linear output layer; a separate target network $Q_{\bar{\theta}}$ supplies the bootstrap, the next-state mask m_{t+1} restricts the inner arg max, and terminal transitions drop the bootstrap so that $y_t = r_t$. Training samples the recorded replay buffers (Adam, smooth- L_1 loss, periodic target sync), and a state falls back to the per-action reward mean only when the network has no evidence for it. The reported learned planner is this trained network, evaluated on a held-out test split. In replay mode, LLM responses come from a content-addressed cache and deterministic executors re-run from fixed seeds, so all planners face the same cached provider behavior and budget limits.

3.3 Prototype Boundary

The implementation follows the MDP boundary. State construction maps generation logs to fixed-length features, masking removes infeasible actions before scoring, selection chooses an action using a deterministic baseline, a contextual bandit, or the trained masked Double-DQN, and executors are the only components that spend budget and change the evidence log. The Python prototype includes a boundary generator from type hints, constants, and simple AST patterns, a property-style fuzzer over inferred input values, cached LLM instruction and assertion executors, oracle refinement through cached LLM calls, counterexample expansion by rerunning failing or seed cases, and repair-context retesting when a repair event is present.

Every executor returns generated tests, observations, cost, provenance, and normalized outcomes. The event log records state features, selected action, action mask, executor observations, token/runtime/fuzzing cost, generated tests, coverage change, failure outcome, oracle uncertainty, reward components, next-state hash, provider/model metadata, and cache mode. Because reward components are stored explicitly, the same log supports audit, aggregation, replay planning, and future neural learners.

We fix one dimension to keep the artifact focused: lower-level executor parameters such as prompt length, mutation budget, and symbolic target selection are held constant, so the planner learns only which top-level action family to select. Because the value function is trained offline from logged baseline trajectories, it is prone to over-estimating the value of actions that are rare in the buffer; we mitigate this with a Conservative Q-Learning penalty [11] (Section 5), which pushes down the value of under-covered actions and, in our held-out evaluation, recovers the cheap optimum where it suffices while leaving regime-dependent adaptation intact. A natural extension is a hierarchy that learns executor parameters below each top-level action without changing logs or masks.

4 Evaluation Design

The evaluation asks when planning changes the outcome, when it changes only the path to the outcome, and when it is unnecessary. RQ1 checks whether planners recover the cheap boundary action on the held-out full-pilot test split. RQ2 asks where planners diverge once duplicate penalties and action-family saturation matter. RQ3 varies token budget and horizon for the Double-DQN, RQ4 inspects action allocation, and RQ5 checks deterministic replay. The questions deliberately separate failure discovery from budget use: two planners can find the same failures while leaving very different evidence traces.

The benchmark contains 74 Python subjects: 20 HumanEval-style tasks, 20 MBPP-style tasks, 10 curated real-bug tasks, 12 stress tasks, and 12 action-balanced tasks. The first three groups form the full-pilot profile, where boundary tests are usually cheap and valid. Stress and action-balanced profiles add duplicate penalties and action-family saturation, creating a controlled contrast

between a regime where simple boundary selection should succeed and regimes where the next useful action may change.

We compare existing tests, a fixed schedule (Static instruction synthesis), random/property fuzzing, a myopic Greedy planner, a contextual bandit (Bandit), a LinUCB bandit with upper-confidence exploration [12] in the spirit of the bandit schedulers used in fuzzing [23,14,6], and the conservative masked Double-DQN. These baselines separate non-adaptive execution, stochastic exploration, reward-per-cost greediness, one-step learned utility, and sequential value learning. The full-pilot profile is split 16/12/22 into training, validation, and test subjects; the learned planner is trained on the 16 training subjects (seeds 0, 1, and 2) and evaluated held-out. We chose the CQL weight from $\{0, 0.3, 0.5, 1, 3\}$ on the held-out results below, not on the validation split (appendix, CQL weight table); the full-pilot test split serves as in-distribution evaluation and the stress and action-balanced profiles as out-of-distribution generalization. The main comparison uses Gemini 2.5 Flash Lite caches; the provider-cache check (RQ5) additionally builds an OpenAI GPT-4o-mini cache. After cache construction, evaluation runs in replay mode. Token budgets are 300/1000/2000 tokens (low/medium/high), horizons are 8/16/32 actions, and the main comparison uses the medium budget at horizon 16. Random/property fuzzing and replay conditions that invoke seeded executors use seeds 0 to 4. Policy decisions are deterministic once the provider cache, trained model, budget, and horizon are fixed.

Primary metrics are failure-found rate, median first-failure step, coverage, duplicate rate, LLM action share, cumulative reward, and action distribution. Coverage counts executed tests, not lines or branches: each executed test adds 0.02 (re-executed inputs included, at most 0.25 per action), capped at 1 per subject. Failure-found rates carry 95% Wilson intervals, comparisons use exact McNemar tests paired by subject, and per-seed Double-DQN results are in the appendix. Median step is right-censored when no failure is found, and action distributions reveal whether gains come from cheap deterministic tests, fuzzing, or LLM-assisted oracle work. Full protocol tables and secondary sensitivity tables are in the separate appendix.

For reproducibility, the artifact fixes subject manifests, train/test splits, budgets, provider caches, planner logs, replay buffers, trained planner JSON files, and aggregation scripts. The Double-DQN is trained from the `replay.json` buffers with Adam, a smooth- L_1 Bellman loss, a periodically synced target network, and a Conservative Q-Learning penalty (weight 0.5); the trained weights are stored as JSON so inference is deterministic and dependency-light. Provider calls use temperature 0 and default `max_tokens=256`. All reported runs replay cached responses, with seeds 0–4 for stochastic or seeded executor paths.

Two protocol controls make the comparison repeatable. First, provider responses are cached before policy comparison, so policies see replayed executor outcomes rather than fresh stochastic completions. The provider rows therefore audit deterministic replay over two cache partitions (Gemini and OpenAI GPT-4o-mini). They are not evidence that live provider calls are interchangeable. Second, all planners share the same action mask, cost accounting, and duplicate

filter. A policy cannot improve by selecting an infeasible action or by sending near-identical tests through the metric pipeline unchecked.

5 Results

Figure 2 and Tables 1–2 report the held-out evaluation. Outcome and allocation must be read together: equal failure-found rates can hide different cost profiles, and higher reward can reflect utility-weighted evidence rather than more bugs.

RQ1: Cheap-action recovery. Where boundary tests are cheap and valid, a good planner should simply use them, and all three learners do. On the full-pilot test split, Greedy, Bandit, and the conservative Double-DQN reach 18.2% failure-found rate, 97.3% coverage, and reward 53.0 (Fig. 2(a), Table 1(a)); 97.3% means that sixteen boundary steps reach the cap on 7 of the 22 subjects and 0.96 on the other 15. The fixed schedule matches the failure-found rate but at lower coverage (17.8%) and a fixed 40.0% LLM share, and random/property fuzzing trails on both. Recovering it is not automatic for a learned planner.

Conservatism ablation. The obstacle is offline over-estimation. Trained from logged trajectories whose coverage is dominated by boundary tests, a vanilla Double-DQN over-estimates the rarely-logged LLM and fuzzing actions: on the test split its estimated Q for LLM-assertion actions exceeds the realized return by +3.8, versus +2.3 for the calibrated boundary action (seed 0; +2.5 and +1.6 on average over three seeds). It then spends $\sim 37\%$ of full-pilot steps on actions that find no failures here, dropping coverage to 68.2% (66.8% to 70.8% across seeds). Conservative Q-Learning [11] corrects this by pushing the under-covered values down: at weight 0.5 the gap falls to +1.3 (+0.3 on average), and the policy recovers the boundary optimum (97.3% coverage, reward 53.0) without losing the adaptation we examine next. Stronger penalties (≥ 1.0) drop the stress failure-found rate from 66.7% to 41.7%; 0.5 spends under half the LLM tokens of 0.3 on stress (186 vs. 417 per subject) at the cost of one action-balanced subject in one seed, so we keep 0.5 (appendix, CQL weight table).

RQ2: Planner divergence. When boundary tests saturate, adaptation pays. On the adversarial profiles, Bandit and the Double-DQN avoid Greedy’s over-commitment: both reach 66.7% failure-found rate on stress subjects versus Greedy’s 25.0%, and 58.3% and 55.6% on action-balanced subjects versus 16.7% (Fig. 2(b–c), Table 1(b)). Per profile the gap to Greedy is not significant (exact McNemar $p = 0.0625$; 0.125 for one action-balanced seed), but pooled over both profiles it is ($p \leq 0.004$). They also earn more reward than Greedy by reallocating toward fuzzing as cheap tests stop paying: the Double-DQN earns 67.6 on stress (from 51.7 to 78.6 across seeds) versus 23.8, and 49.8 on action-balanced (from 33.6 to 61.1) versus 9.9. Static matches the failure-found rate only by running its LLM-heavy schedule on every profile, adaptive or not. LinUCB also matches it on all profiles and earns the highest adversarial reward (114.8, 84.8), with the largest LLM share there (33.3%, 37.7%).

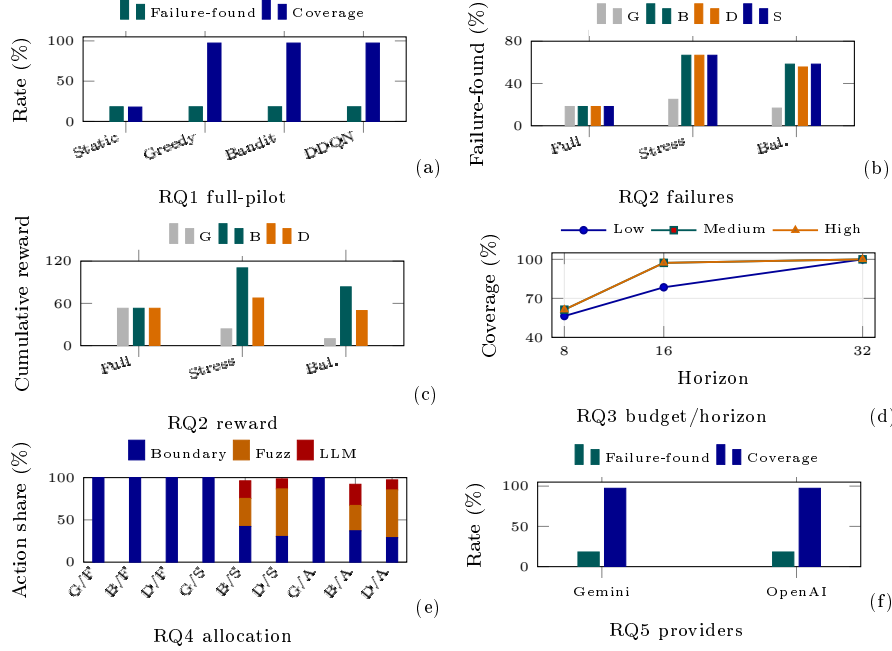


Fig. 2. Main result dashboard for the conservative masked Double-DQN. G: Greedy, B: Bandit, D: Double-DQN, S: Static. F/S/A in panel (e) denote full-pilot, stress, and action-balanced profiles. Panel (f) shows identical aggregates under the Gemini and OpenAI (GPT-4o-mini) cache partitions, confirming deterministic replay (the policy issues no LLM calls on full-pilot, so provider choice cannot change the outcome).

RQ3: Token budget and horizon sensitivity. Horizon helps; the token budget barely binds. The Double-DQN holds an 18.2% failure-found rate while coverage and reward grow with the horizon (reward 26 to 106 from horizon 8 to 32; Fig. 2(d), Table 2(a)). At and above its training budget it spends no LLM tokens on full-pilot, so medium and high budgets agree exactly; only the tightest budget perturbs the state enough to induce mild out-of-distribution token use.

RQ4: Action allocation. Allocation explains these outcomes. Greedy stays on boundary tests throughout; the conservative Double-DQN does too on full-pilot, but on the adversarial profiles it reallocates to fuzzing (~56% share) and some LLM oracle work (Fig. 2(e), Table 2(b)), mirroring Bandit’s shift with a leaner LLM budget on average (the Double-DQN’s LLM share ranges from 1.7% to 32.8% across seeds).

RQ5: Provider-cache replay. Replay is deterministic. The Gemini and OpenAI (GPT-4o-mini) cache partitions yield identical aggregates (Fig. 2(f), Table 2(c)); here the equivalence is also structural, since the conservative policy issues no LLM calls on full-pilot. This audits reproducibility, not live-provider equivalence.

Table 1. Primary held-out outcome and planner-divergence results under the medium budget at horizon 16. The learned planner is a conservative masked Double-DQN (offline CQL, $\alpha=0.5$, mean of 3 seeds) trained on the training split and evaluated on the held-out full-pilot test split ($n=22$) and the out-of-distribution stress and action-balanced profiles ($n=12$ each). Failure-found is the fraction of subjects with at least one failure; 95% Wilson intervals are [7.3, 38.5]% for 4/22, [39.1, 86.2]% for 8/12, [32.0, 80.7]% for 7/12, [25.4, 74.6]% for 6/12, [8.9, 53.2]% for 3/12, and [4.7, 44.8]% for 2/12. Per-seed Double-DQN values are in the appendix.

(a) RQ1 full-pilot held-out test split ($n=22$).						
Method	Fail	Cov.	Return	Boundary	Fuzz	LLM
Existing tests	0.0%	0.0%	0.00	0.0%	0.0%	0.0%
Random/property fuzzing	18.2%	13.1%	0.93	15.9%	17.2%	41.5%
Static instruction synthesis	18.2%	17.8%	2.74	20.0%	20.0%	40.0%
Greedy planner	18.2%	97.3%	53.03	100.0%	0.0%	0.0%
Contextual bandit	18.2%	97.3%	53.03	100.0%	0.0%	0.0%
LinUCB bandit	18.2%	64.3%	48.41	47.4%	8.5%	44.0%
Masked Double-DQN	18.2%	97.3%	53.03	100.0%	0.0%	0.0%

(b) RQ2 compact profile-wise comparison.						
Profile	Method	Fail	Cov.	Boundary	Fuzz	LLM
Full pilot	Static instruction synthesis	18.2%	17.8%	20.0%	20.0%	40.0%
Full pilot	Greedy planner	18.2%	97.3%	100.0%	0.0%	0.0%
Full pilot	Contextual bandit	18.2%	97.3%	100.0%	0.0%	0.0%
Full pilot	LinUCB bandit	18.2%	64.3%	47.4%	8.5%	44.0%
Full pilot	Masked Double-DQN	18.2%	97.3%	100.0%	0.0%	0.0%
Stress	Static instruction synthesis	66.7%	9.3%	20.0%	20.0%	40.0%
Stress	Greedy planner	25.0%	4.3%	100.0%	0.0%	0.0%
Stress	Contextual bandit	66.7%	8.8%	42.2%	33.2%	20.8%
Stress	LinUCB bandit	66.7%	8.8%	35.4%	30.7%	33.3%
Stress	Masked Double-DQN	66.7%	9.3%	30.4%	56.2%	11.9%
Action-balanced	Static instruction synthesis	58.3%	10.5%	20.0%	20.0%	40.0%
Action-balanced	Greedy planner	16.7%	4.5%	100.0%	0.0%	0.0%
Action-balanced	Contextual bandit	58.3%	10.3%	37.1%	30.0%	25.0%
Action-balanced	LinUCB bandit	58.3%	10.3%	30.9%	26.7%	37.7%
Action-balanced	Masked Double-DQN	55.6%	10.6%	29.0%	56.4%	11.9%

6 Discussion

The results argue for a workload-dependent view of adaptive planning. On full-pilot subjects, boundary tests already expose the cheap evidence and the conservative Double-DQN stays with them; on stress and action-balanced subjects, saturation moves boundary tests off the dominant-action ridge and Bandit and the Double-DQN gain roughly 40 failure-rate points over Greedy. Static instruction synthesis is a strong non-adaptive baseline, but its fixed 40% LLM share cannot tell when LLM-assisted work is needed or replaceable by fuzzing.

The learned planner’s calibration matters as much as its architecture. Trained offline, a vanilla Double-DQN over-estimates actions that are rare in the buffer and over-explores them where they find nothing. A mild conservative penalty fixes this without erasing adaptation, while stronger conservatism erodes the adversarial-profile gains, so the penalty is a tunable knob rather than a monotone improvement. On near-expert, short-horizon logged data, Greedy and the bandit

Table 2. Sensitivity, allocation, and provider-cache checks for the conservative masked Double-DQN on the held-out test split. Tok. reports observed LLM-token use as a fraction of the token budget. RQ3/RQ5 use 66 episodes (22 test subjects $\times$ 3 seeds); the Gemini and OpenAI (GPT-4o-mini) partitions give identical aggregates because the policy uses no LLM calls on full-pilot.

(a) RQ3 budget/horizon.					
Setting	Fail	Cov.	Tok.	Reward	Ep.
low/h8	18.2%	56.4%	33.4%	24.63	66
low/h16	18.2%	78.5%	31.2%	52.28	66
low/h32	18.2%	100.0%	31.2%	105.20	66
medium/h8	18.2%	61.3%	0.0%	26.52	66
medium/h16	18.2%	97.3%	0.0%	53.03	66
medium/h32	18.2%	100.0%	0.0%	106.07	66
high/h8	18.2%	61.3%	0.0%	26.52	66
high/h16	18.2%	97.3%	0.0%	53.03	66
high/h32	18.2%	100.0%	0.0%	106.07	66

(b) RQ4 action allocation.					
Profile	Planner		Boundary	Fuzz	LLM
Full pilot	Greedy planner		100.0%	0.0%	0.0%
Full pilot	Contextual bandit		100.0%	0.0%	0.0%
Full pilot	Masked Double-DQN		100.0%	0.0%	0.0%
Stress	Greedy planner		100.0%	0.0%	0.0%
Stress	Contextual bandit		42.2%	33.2%	20.8%
Stress	Masked Double-DQN		30.4%	56.2%	11.9%
Action-balanced	Greedy planner		100.0%	0.0%	0.0%
Action-balanced	Contextual bandit		37.1%	30.0%	25.0%
Action-balanced	Masked Double-DQN		29.0%	56.4%	11.9%

(c) RQ5 provider cache.					
Provider	Ep.	Fail	Cov.	Reward	
Gemini	66	18.2%	97.3%	53.03	
OpenAI	66	18.2%	97.3%	53.03	

are legitimately strong baselines, and the contribution is a planner that matches them where the cheap action is optimal while adapting where it is not.

The action log is therefore part of the main evidence: failure-found rate alone hides whether a planner adapts to state or buys the same outcome through a fixed schedule. Masks play the same role for feasibility, making prerequisite violations (retesting without a repair, refining a certain oracle, calling an LLM with no token budget) impossible before scoring.

The evaluation targets interface validation and controlled planner comparison. The 74-subject Python benchmark is sized for inspection, not repository-scale generality, and with 12 subjects per adversarial profile the per-profile differences are not significant at the 0.05 level; only the pooled comparison is. Because offline value quality depends on trace coverage, outperforming rather than matching the cheap baselines needs richer action coverage (exploratory or online interaction) and a denser failure reward; richer corpora, live-provider runs, repository harnesses, and cross-language tasks are natural next evaluations. For deployment, planning helps most after counterexamples, uncertain oracles, duplicate tests, or repair context change what should happen next.

7 Related Work

LLM test generation. Prior work uses LLMs to generate unit tests, assertions, project-aware suites, and repair feedback [19,24,3,21,25,10]. Program-analysis-guided methods add path sensitivity, property retrieval, and coverage signals [13,26,2], while empirical studies document brittleness and redundancy in generated tests [17]. We take a complementary view: these generators are selectable executors, and the research problem is allocating budget among them.

Adaptive testing and RL. Adaptive fuzzing studies seed scheduling, mutation selection, energy assignment, and budget-sensitive evaluation [23,14,6,15,20]. Learned testing and repair systems use execution feedback to improve generated tests and code [9,22,18,1,7]. These systems typically adapt one mechanism under a fixed action space. TestGenRL chooses among heterogeneous mechanisms after each evidence update, and its action mask handles prerequisite structure that ordinary fuzzing schedulers do not need, such as requiring a counterexample before expansion or a repair event before repair-context retesting.

Bandits and deep RL. Bandit formulations are attractive because many testing rewards are immediate, and our results confirm that a contextual bandit can be competitive at horizon 16. Deep value methods become useful when action value appears through later evidence, such as oracle refinement followed by counterexample expansion. Our planner follows standard masked Double DQN [16,8], with action masks used for prerequisite and budget feasibility. Trained offline from logged trajectories, it inherits the distributional-shift over-estimation that motivates offline RL [5], so we adopt a conservative value penalty [11]: the logged-trajectory setting, not the network, is the source of the calibration problem.

Budgeted evaluation. Fuzzing studies repeatedly show that conclusions change with budget, randomness, and benchmark composition [15,20]. LLM testing inherits the same risk and adds provider variability and token cost. The replay protocol in TestGenRL isolates planner effects from provider sampling effects.

8 Conclusion

TestGenRL formulates LLM-based test generation as budgeted sequential decision making. Its MDP state, action masks, reward components, and trained masked Double-DQN let heterogeneous generators be selected from current evidence rather than from a fixed schedule. The 74-subject Python evaluation shows where this matters: adaptive planners recover the boundary strategy when cheap tests dominate, and they more than double Greedy’s failure-found rate on stress subjects where boundary actions saturate. Trained offline, the planner must be made conservative, or it over-explores actions the logged data under-covers. The interface is ready for richer executors, repository-scale benchmarks, and cross-language evaluation.

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

1. Bouzenia, I., Devanbu, P., Pradel, M.: Repairagent: An autonomous, llm-based agent for program repair. arXiv (2024). <https://doi.org/10.48550/arXiv.2403.17134>
2. Chen, Y., Chen, J., Lin, B., Wang, Z., Zhou, J.: Advancing code coverage: Incorporating program analysis with large language models. arXiv (2024). <https://doi.org/10.48550/arXiv.2404.04966>
3. Chen, Y., Hu, Z., Chen, Z., Han, J., Deng, S., Yin, J.: Chatunitest: A framework for llm-based test generation. arXiv (2023). <https://doi.org/10.48550/arXiv.2305.04764>
4. Eniser, H.F., Wustholz, V., Christakis, M.: Automatically testing functional properties of code translation models. arXiv (2023). <https://doi.org/10.48550/arXiv.2309.12813>
5. Fujimoto, S., Meger, D., Precup, D.: Off-policy deep reinforcement learning without exploration. In: Proceedings of the 36th International Conference on Machine Learning (ICML). vol. 97, pp. 2052–2062 (2019), <https://arxiv.org/abs/1812.02900>
6. Gohil, V., Kande, R., Chen, C., Sadeghi, A.R., Rajendran, J.: Mabfuzz: Multi-armed bandit algorithms for fuzzing processors. Design, Automation & Test in Europe Conference (DATE) (2024), <https://arxiv.org/pdf/2311.14594>
7. Gu, S., Zhang, Q., Li, K., Fang, C., Tian, F., Zhu, L., Zhou, J., Chen, Z.: Testart: Improving llm-based unit testing via co-evolution of automated generation and repair iteration. arXiv (2024). <https://doi.org/10.48550/arXiv.2408.03095>
8. van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 30, pp. 2094–2100 (2016), <https://arxiv.org/abs/1509.06461>
9. Kil, S., Woo, M., Brumley, D.: Program-adaptive mutational fuzzing. OpenAlex (2015), <https://doi.org/10.1109/sp.2015.50>
10. Konstantinou, M., Degiovanni, R., Zhang, J.M., Harman, M., Papadakis, M.: Yate: The role of test repair in llm-based unit test generation. arXiv (2025). <https://doi.org/10.48550/arXiv.2507.18316>
11. Kumar, A., Zhou, A., Tucker, G., Levine, S.: Conservative q-learning for offline reinforcement learning. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 33, pp. 1179–1191 (2020), <https://arxiv.org/abs/2006.04779>
12. Li, L., Chu, W., Langford, J., Schapire, R.E.: A contextual-bandit approach to personalized news article recommendation. In: Proceedings of the 19th International Conference on World Wide Web (WWW) (2010). <https://doi.org/10.1145/1772690.1772758>
13. Liao, D., Yin, X., Pan, S., Ni, C., Xing, Z., Sun, X.: Navigating the labyrinth: Path-sensitive unit test generation with large language models. arXiv (2025). <https://doi.org/10.48550/arXiv.2509.23812>
14. Luo, S., Herrera, A., Quirk, P., Chase, M., Ranasinghe, D.C., Kanhere, S.S.: Make out like a (multi-armed) bandit: Improving the odds of fuzzer seed scheduling with t-scheduler. arXiv preprint arXiv:2312.04749 (2023), <https://arxiv.org/pdf/2312.04749>

15. Manes, V.J.M., Han, H.S., Han, C., Kil, S., Egele, M., Schwartz, E.J., Woo, M.: The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering* (2019), <http://arxiv.org/abs/1812.00140>
16. Mnih, V., Kavukcuoglu, K., Silver, D., et al.: Human-level control through deep reinforcement learning. *Nature* **518**(7540), 529–533 (2015). <https://doi.org/10.1038/nature14236>
17. Ouedraogo, W.C., Li, Y., Dang, X., Tang, X., Koyuncu, A., Klein, J., Lo, D., Bissyande, T.F.: Test smells in llm-generated unit tests. *arXiv* (2024). <https://doi.org/10.48550/arXiv.2410.10628>
18. Prasad, A., Stengel-Eskin, E., Chen, J.C.Y., Khan, Z., Bansal, M.: Learning to generate unit tests for automated debugging. *arXiv* (2025). <https://doi.org/10.48550/arXiv.2502.01619>
19. Schafer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. *arXiv* (2023). <https://doi.org/10.48550/arXiv.2302.06527>
20. Schloegel, M., Bars, N., Schiller, N., Bernhard, L., Scharnowski, T., Crump, A., Ale-Ebrahim, A., Bissantz, N., Muench, M., Holz, T.: Sok: Prudent evaluation practices for fuzzing. *OpenAlex* (2024), <http://arxiv.org/abs/2405.10220>
21. Wang, H., Hu, H., Chen, C., Turhan, B.: Chat-like asserts prediction with the support of large language model. *arXiv* (2024). <https://doi.org/10.48550/arXiv.2407.21429>
22. Wang, J., Song, C., Yin, H.: Reinforcement learning-based hierarchical seed scheduling for greybox fuzzing. *OpenAlex* (2021), <https://doi.org/10.14722/ndss.2021.24486>
23. Yue, T., Wang, P., Tang, Y., Wang, E., Yu, B., Lu, K., Zhou, X.: Ecofuzz: Adaptive energy-saving greybox fuzzing as a variant of the adversarial multi-armed bandit. 29th USENIX Security Symposium (2020), <https://www.usenix.org/system/files/sec20-yue.pdf>
24. Zapkus, D.M., Slotkiene, A.: Unit test generation using large language models: A systematic literature review. *Vilnius University Open Series* (2024). <https://doi.org/10.15388/lmitt.2024.20>
25. Zhang, Y., Lu, Q.Y., Liu, K., Dou, W., Zhu, J., Li, Q., Zhang, C., Zheng, L., Wei, J.: Citywalk: Enhancing llm-based c++ unit test generation via project-dependency awareness and language-specific knowledge. *arXiv* (2025). <https://doi.org/10.48550/arXiv.2501.16155>
26. Zhang, Z., Liu, X., Lin, Y., Gao, X., Sun, H., Yuan, Y.: Llm-based unit test generation via property retrieval. *arXiv* (2024). <https://doi.org/10.48550/arXiv.2410.13542>